

Week 5: Databases

MY472: Data for Data Scientists

https://lse-my472.github.io/

Autumn Term 2026

Ryan Hübert

Associate Professor of Methodology

Slides last updated: 27 October 2026

Outline

1. Transforming, summarising and manipulating data
2. What *should* tabular data look like?
3. Merging and joining
4. Relational databases

Sales data

Recall our sales data from last week

```
print(sales)
```

```
     variety  colour  kg_sold  gbp_per_kg        date
0      Donut  Yellow      150        4.40  2026-07-01
1  Nectarine     Red      100        7.75  2026-07-01
2      White   White      120        5.00  2026-07-01
3      Donut  Yellow      140        4.00  2026-08-01
4  Nectarine     Red      200        7.50  2026-08-01
5      White   White       60        5.10  2026-08-01
```

Wrangling data in pandas

To work with data in pandas, manipulate columns directly:

```
# add a new variable
sales["revenue"] = sales["kg_sold"] * sales["gbp_per_kg"]
# keep only these columns
sales = sales[["variety", "revenue"]]
# sort by revenue in descending order
sales = sales.sort_values("revenue", ascending = False)

print(sales.head())
```

```
     variety  revenue
4  Nectarine   1500.0
1  Nectarine    775.0
0      Donut    660.0
2      White    600.0
3      Donut    560.0
```

Filtering data frame rows

Remember: `.loc[rows, columns]` selects by label (last week)

```
print(sales.loc[[0, 3], ["variety", "kg_sold"]])
```

```
  variety  kg_sold
0   Donut      150
3   Donut      140
```

Usually you don’t know which row labels you want, only a condition (e.g. sold over 100 kg)

Filtering data frame rows

Remember, in Python, a comparison like `150 > 100` gives a `bool`, which is either `True` or `False`

You can use these comparisons on entire columns

- You get a series with dtype `bool` that contains a `True` or `False` (or `NaN`) per row

```
print(sales["kg_sold"] > 100)
```

```
0     True
1    False
2     True
3     True
4     True
5    False
Name: kg_sold, dtype: bool
```

A series with a `bool` dtype that's the same length as a data frame is called a **mask** because it "masks" certain rows

Filtering data frame rows

Use `.loc[]` with a mask to keep rows where the mask returns `True`

```
mask = sales["kg_sold"] > 100
print(sales.loc[mask, :])
```

```
     variety  colour  kg_sold  gbp_per_kg        date
0      Donut  Yellow      150         4.4  2026-07-01
2      White   White      120         5.0  2026-07-01
3      Donut  Yellow      140         4.0  2026-08-01
4  Nectarine     Red      200         7.5  2026-08-01
```

Rows labelled 1 and 5 are removed—their mask value is `False`

This is often referred to as **filtering** the data (by rows)

Method chaining

Method chaining allows us to do several operations in one expression:

```
sales = pd.read_csv("data/peach_sales.csv")
sales["revenue"] = sales["kg_sold"] * sales["gbp_per_kg"]
revenue = (sales[["variety", "revenue"]]
             .sort_values("revenue", ascending = False))
print(revenue)
```

```
     variety  revenue
4  Nectarine   1500.0
1  Nectarine    775.0
0      Donut    660.0
2      White    600.0
3      Donut    560.0
5      White    306.0
```

Note: you must surround the chain in parentheses

Descriptive statistics

`.describe()` summarises every numerical column at once

```
print(sales.describe())
```

```
          kg_sold  gbp_per_kg      revenue
count    6.000000     6.00000     6.000000
mean   128.333333     5.62500   733.500000
std     47.504386     1.60242   406.282537
min     60.000000     4.00000   306.000000
25%    105.000000     4.55000   570.000000
50%    130.000000     5.05000   630.000000
75%    147.500000     6.90000   746.250000
max    200.000000     7.75000  1500.000000
```

For a categorical column, ask for counts instead

```
print(sales["variety"].value_counts())
```

```
variety
Donut        2
Nectarine    2
White        2
Name: count, dtype: int64
```

Descriptive statistics

Can apply methods separately to numerical columns: `.sum()`, `.mean()`, `.max()`, etc.

```
print(sales.max())
```

```
variety            White
colour            Yellow
kg_sold              200
gbp_per_kg          7.75
date          2026-08-01
revenue           1500.0
dtype: object
```

And to individual columns:

```
print(sales["kg_sold"].max())
```

```
200
```

Grouping in pandas

Sometimes data has a “nested” structure, such as:

- each observation is *nested* under a single unit
- each unit is *nested* under a higher-level unit (cluster)
- each observation is *nested* under a bin based on a variable

Example: peach seller has a dataset of 100 peaches’ weights

```
print(peach_weights.head()) # show first five rows
```

```
     variety  weight_g
0      White       187
1      Donut       162
2      Donut       167
3  Nectarine       174
4  Nectarine       172
```

In this data: a unit is a peach, and each unit is clustered by variety

Grouping in pandas

Can group by a higher variable and summarise across that variable:

```
# Group by variety and summarise:
pw_summary = (peach_weights
                .groupby("variety")
                .agg(
                  count = ("weight_g", "size"),
                  total_weight = ("weight_g", "sum"),
                  mean_weight = ("weight_g", "mean")
                ))
print(pw_summary)
```

```
           count  total_weight  mean_weight
variety
Donut         31          5572   179.741935
Nectarine     42          7115   169.404762
White         27          4866   180.222222
```

Grouping in pandas

Note what `groupby()` did: `variety` is now the index, not a column

- To get an ordinary table back, ask for the index to be reset

```
print(pw_summary.reset_index())
```

```
     variety  count  total_weight  mean_weight
0      Donut     31          5572   179.741935
1  Nectarine     42          7115   169.404762
2      White     27          4866   180.222222
```

Outline

1. Transforming, summarising and manipulating data
2. What *should* tabular data look like?
3. Merging and joining
4. Relational databases

What's wrong with this table?

Another peach seller sends you his record of kilos sold:

	variety	2026-07-01	2026-08-01
0	Donut	150	140
1	Nectarine	100	200
2	White	120	60

A problem: to do analysis (e.g. total the kilos sold), you must write each date column by hand:

```
data[["2026-07-01", "2026-08-01"]].sum().sum()
```

```
np.int64(770)
```

Inconvenient on its own, but when you add more data (e.g., September) into a new column, the analysis code is now wrong

Column names are data: `2026-07-01` is a value of a variable!

What *should* tabular data look like?

This is an example of a more general phenomenon

- Lots of time is spent cleaning (tabular) data
- Why?
  - It needs to be used for analysis
  - But tabular data is not often created with analysis in mind
  - You get what you get!

There are a set of principles that can ensure tabular data is in its “best” form

Tidy data

Hadley Wickham's 2014 article "Tidy Data" (in *Journal of Statistical Software* 59(10)) suggests a "best" form

Tidy data is data that follows three rules:

1. Each variable is a column
2. Each observation is a row
3. Each type of observational unit is a table

One column per thing you measured, one row per thing you measured it on, one table per kind of thing

Fixing columns that are values

To “tidy” the untidy sales data, **melt** columns using `.melt()`

```
print(data.melt(
  id_vars = "variety",
  value_vars = ["2026-07-01", "2026-08-01"],
  var_name = "date",
  value_name = "kg_sold"))
```

```
     variety        date  kg_sold
0      Donut  2026-07-01      150
1  Nectarine  2026-07-01      100
2      White  2026-07-01      120
3      Donut  2026-08-01      140
4  Nectarine  2026-08-01      200
5      White  2026-08-01       60
```

This takes us from **wide** to **long** format

Observations split across rows

That's not the only way data can be untidy

Suppose you get some data from yet another seller:

```
print(data2.head())
```

```
     variety  colour        date variable  value
0      Donut  Yellow  2026-07-01  kg_sold  150.0
1  Nectarine     Red  2026-07-01  kg_sold  100.0
2      White   White  2026-07-01  kg_sold  120.0
3      Donut  Yellow  2026-08-01  kg_sold  140.0
4  Nectarine     Red  2026-08-01  kg_sold  200.0
```

Here: observations scattered across multiple rows

Fixing rows that should be columns

To “tidy” this data **pivot** those rows into a new pair of columns using `.pivot()`

```
tidy2 = data2.pivot(
  index = ["variety", "colour", "date"],
  columns = "variable",
  values = "value").reset_index()
tidy2.columns.name = None   # `pandas` kept 'var' as a label
print(tidy2)
```

```
     variety  colour        date  gbp_per_kg  kg_sold
0      Donut  Yellow  2026-07-01         4.40    150.0
1      Donut  Yellow  2026-08-01         4.00    140.0
2  Nectarine     Red  2026-07-01         7.75    100.0
3  Nectarine     Red  2026-08-01         7.50    200.0
4      White   White  2026-07-01         5.00    120.0
5      White   White  2026-08-01         5.10     60.0
```

This takes us from long to wide format

What does “tidy” data look like in Python?

Is the seller’s original sales data tidy?

```
print(sales)
```

```
     variety  colour  kg_sold  gbp_per_kg        date
0      Donut  Yellow      150        4.40  2026-07-01
1  Nectarine     Red      100        7.75  2026-07-01
2      White   White      120        5.00  2026-07-01
3      Donut  Yellow      140        4.00  2026-08-01
4  Nectarine     Red      200        7.50  2026-08-01
5      White   White       60        5.10  2026-08-01
```

Assume all peaches of a variety have same colour, then:

- `sales` follows rules 1 and 2, but not rule 3
- Variety colour is repeated across rows
- `colour` is a fact about *variety* not *sales*

Storing a fact multiple times

Rule 3 guards against repeating the same information multiple times in a table

Three things go wrong:

1. **Update anomaly**: correct Donut's colour in one row, miss another, and the table gives two answers to the same question
2. **Insertion anomaly**: you cannot record that a new variety is white until somebody buys one
3. **Deletion anomaly**: delete the last Donut sale and you lose the fact that Donuts are yellow

Storing each fact *once* is how you prevent these

Splitting the peach data

Store the colour of each variety once, in its *own* table

```
varieties = pd.DataFrame({"variety": ["Donut", "Nectarine", "White"],
                          "colour": ["Yellow", "Red", "White"]})
print(varieties)
```

```
     variety  colour
0      Donut  Yellow
1  Nectarine     Red
2      White   White
```

```
sales = sales.drop(columns = "colour")   # now stored in varieties
print(sales)
```

```
     variety  kg_sold  gbp_per_kg        date
0      Donut      150        4.40  2026-07-01
1  Nectarine      100        7.75  2026-07-01
2      White      120        5.00  2026-07-01
3      Donut      140        4.00  2026-08-01
4  Nectarine      200        7.50  2026-08-01
5      White       60        5.10  2026-08-01
```

Primary keys: `variety` in `varieties`, `variety`+`date` in `sales`

Normalisation

Organising data so each fact is stored once is known as **normalisation**

- Ideas developed initially by E. F. Codd (see Wikipedia page)

A bit complicated, but three main rules, applied in order:

1. Each cell in a table contains a single value
   - This is very intuitive (no examples so far!)
2. Every column states a fact about the whole primary key
3. A fact about something other than the key goes in its own table

Tidy data in practice

Most analysis tools assume rules 1 and 2: one row per observation, one column per variable

- `groupby()`, `.merge()`, plotting, regression: all expect that shape

Most data you download does not have it

- World Bank: one column per year
- Eurostat: one column per time period

Rule 3 is different: it splits one table into several, and analysis needs them back in one

- Store the data in separate tables; join them when you analyse

Always know what one row is in each table

Outline

1 Transforming, summarising and manipulating data

2 What *should* tabular data look like?

3 Merging and joining

4 Relational databases

Merges and joins

A **join** (or **merge**) combines two tables into one

- Matches rows on "join keys": columns appearing in both tables

A join creates a new table with columns from tables being joined, but they differ on which *rows* end up in the new table:

- **Inner join**: keeps only rows with matching keys in both tables
- **Left (right) join**: keeps all rows from the left (right) table, and any matching rows from the right (left) table
- **Full join**: keeps all rows from both tables

If a key repeats in other table, a row is duplicated once per match

Always check your data after joins/merges!

Joining in `pandas`

Recall when we separated `sales` and `varieties`

- You can join them back together with `.merge()`

```
print(sales.merge(varieties, on = "variety", how = "left"))
```

```
     variety  kg_sold  gbp_per_kg        date  colour
0      Donut      150        4.40  2026-07-01  Yellow
1  Nectarine      100        7.75  2026-07-01     Red
2      White      120        5.00  2026-07-01   White
3      Donut      140        4.00  2026-08-01  Yellow
4  Nectarine      200        7.50  2026-08-01     Red
5      White       60        5.10  2026-08-01   White
```

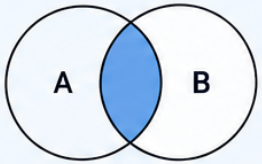
For this join:

- The (shared) key is `variety`, which is in both tables
- A left join (with `sales` on left) makes most sense here (why?)

Joining in pandas

INNER JOIN

Rows with matching keys in both DataFrames




```
pd.merge(A, B, on='key', how='inner')
```

A

key	A_val
1	A1
2	A2
3	A3

B

key	B_val
2	B2
3	B3
4	B4

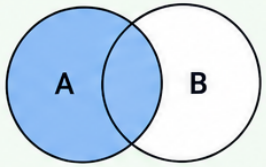
→

Result

key	A_val	B_val
2	A2	B2
3	A3	B3

LEFT JOIN

All rows from left DataFrame (A), matches from right (B)




```
pd.merge(A, B, on='key', how='left')
```

A

key	A_val
1	A1
2	A2
3	A3

B

key	B_val
2	B2
3	B3
4	B4

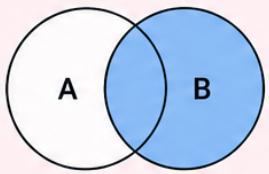
→

Result

key	A_val	B_val
1	A1	NaN
2	A2	B2
3	A3	B3

RIGHT JOIN

All rows from right DataFrame (B), matches from left (A)




```
pd.merge(A, B, on='key', how='right')
```

A

key	A_val
1	A1
2	A2
3	A3

B

key	B_val
2	B2
3	B3
4	B4

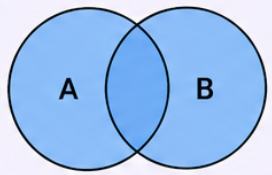
→

Result

key	A_val	B_val
2	A2	B2
3	A3	B3
4	NaN	B4

FULL OUTER JOIN

All rows from both DataFrames (A and B)




```
pd.merge(A, B, on='key', how='outer')
```

A

key	A_val
1	A1
2	A2
3	A3

B

key	B_val
2	B2
3	B3
4	B4

→

Result

key	A_val	B_val
1	A1	NaN
2	A2	B2
3	A3	B3
4	NaN	B4

Joining in pandas

Consider another example: the seller has a chart with culinary qualities of different peach varieties, which she enters into Python

```
peach_features = pd.DataFrame({
  "Variety": ["Donut", "Redhaven", "White"],
  "Taste": ["Tangy", "Sweet", "Sweet"],
  "Fuzziness": ["Fuzzy", "Fuzzy", "Fuzzy"]
})
print(peach_features)
```

```
    Variety  Taste Fuzziness
0     Donut  Tangy     Fuzzy
1  Redhaven  Sweet     Fuzzy
2     White  Sweet     Fuzzy
```

Suppose she eventually wants to analyse how her sales of peach varieties correlates with culinary features

Joining in pandas

So, she needs to merge the peach feature data into her sales data

- The key is `variety`, which is in both tables
- A left join (with `sales` on left) makes most sense here (why?)

```
print(sales[["variety", "kg_sold", "gbp_per_kg"]]
  .merge(peach_features, left_on = "variety", right_on = "Variety",
         how = "left", indicator = True))
```

```
     variety  kg_sold  gbp_per_kg Variety  Taste Fuzziness     _merge
0      Donut      150        4.40   Donut  Tangy     Fuzzy       both
1  Nectarine      100        7.75     NaN    NaN       NaN  left_only
2      White      120        5.00   White  Sweet     Fuzzy       both
3      Donut      140        4.00   Donut  Tangy     Fuzzy       both
4  Nectarine      200        7.50     NaN    NaN       NaN  left_only
5      White       60        5.10   White  Sweet     Fuzzy       both
```

`indicator = True` adds a `_merge` column: `both` where the key matched, `left_only` where it did not

Duplicate join keys

Suppose `varieties` lists Donut twice, with two different colours

- `variety` is no longer a primary key of `varieties`

```
varieties_bad = pd.DataFrame({
  "variety": ["Donut", "Donut", "Nectarine", "White"],
  "colour": ["Yellow", "Orange", "Red", "White"]})
joined = sales.merge(varieties_bad, on = "variety", how = "left")
print(len(sales), len(joined))
```

```
6 8
```

```
print(joined.loc[joined["variety"] == "Donut", :])
```

```
  variety  kg_sold  gbp_per_kg        date  colour
0   Donut      150         4.4  2026-07-01  Yellow
1   Donut      150         4.4  2026-07-01  Orange
4   Donut      140         4.0  2026-08-01  Yellow
5   Donut      140         4.0  2026-08-01  Orange
```

Each Donut sale now appears twice, once per colour

- Now: Donut peach sales are double counted!

Outline

1. Transforming, summarising and manipulating data
2. What *should* tabular data look like?
3. Merging and joining
4. Relational databases

Databases

Last section: we split `sales` data across linked tables

- Need to store tables and keep links between them correct

A **database** is a collection of data that's managed by a program, called a **database management system** (DBMS)

- You do not open the data files yourself
- Send the DBMS a request (a **query**), and it returns a table
- Examples: SQLite, PostgreSQL, MySQL

Could instead have a folder of CSV files, but why not?

- Efficient: only query the data you actually need
- Concurrency: many people can read and write at once

Relational databases

A **relational database** stores normalised tables, linked by keys

- Each table has a primary key, which identifies each row
- **Foreign key**: a column that's the primary key of another table

The peach data as a relational database:

Table	Columns	Primary key	Foreign key
`varieties`	`variety`, `colour`	`variety`	none
`sales`	`variety`, `date`, `kg_sold`, `gbp_per_kg`	`variety` and `date`	`variety`

- `variety` in `sales` is a foreign key, because each sale names a variety in `varieties`

The tables, their columns and their keys: the database's **schema**

Non-relational databases

Some databases do not store tables

- **Non-relational databases** (also called **NoSQL**) store data in other forms, e.g. documents or key-value pairs
- Used when records don't share same columns
- Example: MongoDB

Most social science data is tabular, so in this course we will focus on relational databases only

To get a table out of a relational database, write a query in SQL

SQL: Structured Query Language

SQL is a domain specific language (DSL) designed to define, control access to, manipulate, and query relational databases

- Pronounced both “S-Q-L” and “SEQUEL”

Unlike Python, it is a **nonprocedural/declarative language**: user defines what to do, inputs, and outputs, but not the control flow

- How the statement is executed is left to the **optimiser**, which is opaque to the user
- How long a query takes depends on the optimiser
- Performance will vary, but generally faster than standard data frame manipulation in `pandas` (and much more scalable)

Some common components of SQL queries

An **SQL query** is a command to create a table from a database that meets conditions

Always required in any SQL query:

- `SELECT` columns
- `FROM` a table in a database

Other useful commands:

- `WHERE` rows meet a condition
- `GROUP BY` values of a column
- `ORDER BY` values of a column when displaying results
- `LIMIT` to only X number of rows in resulting table
- “Operators” such as `SUM`, `COUNT`, `AVG`, etc.

Writing to a relational database

SQLite keeps an entire database in a single `.db` file

Use `sqlite3` from the Python standard library:

- Establish a “connection” to the database
- Then write or read data from the database

Managing the connection is *important*:

- Open connection with `sqlite3.connect()`
- Close it with `.close()`
- Prevents data corruption

```
import sqlite3
con = sqlite3.connect("data/peaches.db")
# Do some stuff
con.close()
```

Writing to a relational database

First, let's write a new SQLite database:

```
import sqlite3
con = sqlite3.connect("data/peaches.db")
sales.to_sql("sales", con, if_exists = "replace", index = False)
```

```
varieties.to_sql("varieties", con, if_exists = "replace",
                 index = False)
```

```
con.close()
```

Note:

- When you use `sqlite3.connect()` it will create a new file if one already doesn't exist
- `.to_sql()` saves changes directly to the database
- Do not forget to close the connection!

Reading from a relational database

When you already have a database, you usually want to start by getting a clear idea what is in it

SQLite databases have a table called `sqlite_master` listing contents:

```
con = sqlite3.connect("data/peaches.db")
print(pd.read_sql_query(
      "SELECT name FROM sqlite_master WHERE type = 'table';", con))
```

```
         name
0       sales
1  varieties
```

```
print(pd.read_sql_query("SELECT * FROM sales LIMIT 3;", con))
```

```
     variety  kg_sold  gbp_per_kg        date
0      Donut      150        4.40  2026-07-01
1  Nectarine      100        7.75  2026-07-01
2      White      120        5.00  2026-07-01
```

```
con.close()
```

`LIMIT 3` returns the first three rows

Reading from a relational database

```
con = sqlite3.connect("data/peaches.db")
sales = pd.read_sql_query("SELECT * FROM sales;", con)
print(sales)
```

```
     variety  kg_sold  gbp_per_kg        date
0      Donut      150        4.40  2026-07-01
1  Nectarine      100        7.75  2026-07-01
2      White      120        5.00  2026-07-01
3      Donut      140        4.00  2026-08-01
4  Nectarine      200        7.50  2026-08-01
5      White       60        5.10  2026-08-01
```

```
varieties = pd.read_sql_query("SELECT * FROM varieties;", con)
print(varieties)
```

```
     variety  colour
0      Donut  Yellow
1  Nectarine     Red
2      White   White
```

```
con.close()
```

Reading from a relational database

Can do manipulations directly in a database with SQL queries

```
con = sqlite3.connect("data/peaches.db")
my_query = """
  SELECT s.variety, v.colour, SUM(s.kg_sold) AS kg
  FROM sales AS s LEFT JOIN varieties AS v
  ON s.variety = v.variety
  GROUP BY s.variety, v.colour;
"""
print(pd.read_sql_query(my_query, con))
```

```
     variety  colour   kg
0      Donut  Yellow  290
1  Nectarine     Red  300
2      White   White  180
```

```
con.close()
```

Usually more efficient: only pulls data you need at the moment

SQL and `pandas`

In general: SQL and `pandas` can do same operations on tables

Operation	`pandas`	SQL
Keep some columns	`df[["a", "b"]]`	`SELECT a, b`
Keep some rows	`df.loc[mask, :]`	`WHERE`
Sort rows	`.sort_values()`	`ORDER BY`
Summarise groups	`.groupby().agg()`	`GROUP BY`
Join tables	`.merge()`	`JOIN`

In this course, you write just enough SQL to look inside a database and pull whole tables out of it into `pandas`:

- `SELECT name FROM sqlite_master WHERE type = 'table'`
- `SELECT * FROM table LIMIT 5` (preview table)
- `SELECT * FROM table` (get whole table)

Asking a model to write SQL

The queries above: where did they come from?

A seasoned database engineer would be able to write SQL queries from memory—this is a very common data science task

Now: LLMs can help a lot, but give it the schema, not the data

- Give it table names, column names and types, along with a brief description of what you want
- It never needs rows, and you should *not* paste them in

Once you get a query:

- Use it in Python using `sqlite3` and `pandas`
- Carefully check the output you get